Mini Project

Mid-evaluation Report

Prepared by

Kushagra Gupta

(2025909001)

Under the guidance of

Prof. Vasudeva Varma

Prof. Vandan Mujadia

Submitted in
partial fulfillment of the requirements
for the course of

CL3410: Language Models and Agents



INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY

HYDERABAD

(August 2025)

1. OBJECTIVE

The goal of this project is to build a clean, high-quality multilingual. The selected languages are:

- English (50% of total tokens)
- Hindi (30–40% of total tokens, Mother tongue)
- Manipuri / Bengali (10–20% of total tokens, Indian language)

Since Manipuri corpus resources were found to be extremely limited, **Bengali language** has been selected as a substitute because Manipuri is written in the Bengali script and both languages share **strong script-level similarity**.

2. WORK COMPLETED TILL NOW

2.1 Corpus Preparation

- Collected raw corpora for English, Hindi and Bengali.
- Verified that the required number of tokens is sufficient and have collected additional tokens.
- Merged language-specific corpora into *_corpus.txt file.

2.2 Deduplication

- Automated deduplication pipeline using shell scripting.
- Used `sort –parallel –u` on terminal as it is most efficient and the order doesn't matter for training the model.
- Run on all the 3 languages and Bengali without manual monitoring.
- As corpus is collected from single source, deduplication didn't help that much, and statistics doesn't show any effectiveness.
  Quantitative Reduction Stats: Reduced by ~2% in English, ~3% in Hindi, negligible in Bengali

2.3 Cleaning Pipeline

After sampling text from different parts of the corpus, I figured out, what type of cleaning is needed to be done. Text-cleaning pipeline for **Bengali dataset** included:

1. Unicode normalization (NFC).
2. Remove HTML tags.
3. Remove URLs.
4. Remove emails.
5. Remove emojis (Unicode + colon emoji patterns).
6. Remove || separators.
7. Remove lines with .... or dot-heavy lines.
8. Remove stray single-character Bengali letters.
9. Remove all junk symbols outside Bengali script, English letters, digits, and basic punctuation.
10. Collapse repeated punctuation.

11. Collapse multiple spaces.
12. Remove lines that contain only punctuation.
13. Remove lines that are not mostly Bengali (threshold 50%).
14. Sentence segmentation by Bengali full stops: । and ।.
15. Keep numbers/dates and normal Bengali sentences/poetry.

Same procedure for English. Text-cleaning pipeline for **English dataset** included:
1. Unicode normalization (NFKC).
2. Remove non-printable characters
3. Remove HTML tags.
4. Remove URLs.
5. Remove emails.
6. Remove special junk symbols.
7. Collapse repeated punctuation.
8. Collapse multiple spaces.
9. Remove emojis.
10. Trim whitespace.
11. Skip lines that are only punctuation.
12. Language detection (langid): Discard if detected language ≠ English and confidence > 0.9.
13. Word frequency filtering (wordfreq): Keeps only tokens with zipf_frequency > 1.0
14. Fallback safeguard.
15. Sentence segmentation (NLTK sent_tokenize).

And at last, for Hindi. Text-cleaning pipeline for **Hindi dataset** included:
1. Unicode normalization (NFC)
2. Numbers normalization
3. Remove HTML tags
4. Remove URLs
5. Remove email addresses
6. Remove emojis
7. Dot-heavy filter
8. English-heavy filter
9. Sentence segmentation: splits on Hindi danda ।, ?, !, and single/multiple ..
10. Final Devanagari check before writing.

2.4 Data splits

- Truncated corpus individually for English, Hindi and Bengali (1.65B, 1.15B and 600M) after shuffling. Total 3.4B tokens as 10%-15% is for safety.
- Then splitting the dataset: **90%** train / **5%** test / **5%** validation

2.5 Tokenization

For tokenization, I used **SentencePiece** with **Byte Pair Encoding (BPE)**. This approach works well for multilingual text because it:

- Handles all scripts (English, Hindi, Bengali) without relying on spaces.
- Supports subword-level representation, reducing unknown words.
- Allows a single joint tokenizer for all three languages.

I trained the tokenizer on a small multilingual subset (10,000 lines):

- English: 5,000 lines
- Hindi: 2,000 lines
- Bengali: 3,000 lines

The vocabulary size was set to **16,000** with **full character coverage**, so all characters from English, Hindi, and Bengali are included.

Results:

- Model files created: multilang_tokenizer.model and multilang_tokenizer.vocab
- **Token coverage: 100%** (No <unk> tokens)

Next Step:
Scale this tokenizer to the full corpus (~3B tokens) with a larger vocabulary (32k–50k) for pretraining the multilingual language model.
This ensures good coverage across all three languages and reduces unknown tokens.

3. CHALLENGES AND DECISIONS

- Manipuri Data Scarcity: Very small usable corpus after cleaning and deduplication. To ensure sufficient data coverage, Bengali corpus is used as a substitute since it shares the same script.
- Deduplication Effectiveness: Minimal reduction due to single-source collection.
- Translation models not working effectively: As I have tried alternatives, it is just taking too much time, or the Api is getting crashed. It needs too much of time or rather computation power to do that much translation.

4. DATA STATISTICS

Raw Corpus Size

- English: ~48.5M lines (~1.8B tokens)
- Hindi: ~22.6M lines (~1.2B tokens)
- Bengali (used instead of Manipuri): ~39.8M lines (~900M tokens)
- Total: ~110.9M lines (~3.9B tokens)

After Deduplication

- English: ~42.1M lines (~1.7B tokens)

- Hindi: ~19.8M lines (~1.12B tokens)
- Bengali: ~18M lines (~870M tokens)
- Total: ~42.1 lines (~3.69B tokens)

After Cleaning Pipeline

- English: ~98M lines (~1.7B tokens)
- Hindi: ~67M lines (1.16B~ tokens)
- Bengali: ~20.5M lines (~900M tokens)
- Final Total: ~185.5M lines (~3.76B tokens)

Required for Tokenization

- English: ~50% (~1.5B tokens)
- Hindi: ~35% (~1.05B tokens)
- Bengali: ~15% (~450M tokens)
- Final Total: ~3B tokens

Final Truncated Dataset after shuffling (for tokenization)

- English: ~92M lines (~1.65B tokens)
- Hindi: ~69M lines (~1.15B tokens)
- Bengali: ~13.6M lines (~600M tokens)
- Final Total: ~174.6M lines (~3.4B tokens)

Split Dataset

- Train: ~157M lines (~3B tokens)
- Test: ~8.75M lines (~169M tokens)
- Validation: ~8.76M lines (~170M tokens)
- Final Total: ~174.5M lines (~3.4B tokens)

5. NEXT STEP (FUTURE WORK)

- Train a joint tokenizer (BPE or SentencePiece) on the combined corpus.
- Implement a 100M–150M parameter autoregressive LM.
- Train on the multilingual corpus using PyTorch.
- Fine-tune the model according to the tasks.
    - Counting syllables in a word.
    - Replacing character at some index.
- Evaluating accuracy of the model.

6. TIMELINE

6.1 Phase 1

- Data Collection by August 26, 2025.

- Pre-processing & Cleaning by August 29, 2025.

6.2 Phase 2

- Tokenizer training by September 1, 2025.

6.3 Phase 3

- Model pretraining (100M–150M LM) between September 1-10, 2025

6.4 Phase 4

- Fine-tuning for reasoning tasks between September 10-13, 2025.

6.5 Phase 5

- Evaluation + final report writing by September 15, 2025

7. REFERENCES

7.1 DATASET

The datasets used in this project were collected from multiple open-source repositories and benchmarks:

- English & Hindi
  - AI Bharat Sangrah: https://huggingface.co/datasets/ai4bharat/sangraha
- Bengali
  - AI4Bharat IndicCorpV2: https://huggingface.co/datasets/ai4bharat/IndicCorpV2
- Manipuri
  - AI Bharat Sangrah: https://huggingface.co/datasets/ai4bharat/sangraha
  - AI4Bharat IndicCorpV2: https://huggingface.co/datasets/ai4bharat/IndicCorpV2
  - HuggingFace FineWeb-2: https://huggingface.co/datasets/HuggingFaceFW/fineweb-2
  - WMT Indic MT Task:
    - WMT23: https://www2.statmt.org/wmt23/indic-mt-task.html
    - WMT24: https://www2.statmt.org/wmt24/indic-mt-task.html
    - WMT25: https://www2.statmt.org/wmt25/indic-mt-task.html
  - OPUS Parallel Corpora: https://opus.nlpl.eu/results/en&mni/corpus-result-table
  - Indian Parallel Corpus (GitHub): https://github.com/Kartikaggarwal98/Indian_ParallelCorpus?tab=readme-ov-file
  - Kaggle English–Manipuri Dataset: https://www.kaggle.com/datasets/jigarpanjiyar/english-to-manipuri-parallel-dataset?select=1_manipuri+%28bengali+Scirpt%29.txt

7.2 TRANSLATION

To enable English–Manipuri and Hindi–Manipuri translations, I explored several existing

open-source models and APIs:

- IndicTrans2 Models (AI4Bharat) [link]
  - ai4bharat/indictrans2-en-indic-1B
  - ai4bharat/indictrans2-indic-indic-1B
- Custom Hindi → Manipuri Model: https://huggingface.co/abhishek2475/mb-Hindi-to-manipuri
- IIIT Hyderabad API [Link]
  - SSMT OneMT API: https://ssmt.iiit.ac.in/onemt

7.3 TOKENIZATION

- Google SentencePiece: https://github.com/google/sentencepiece

-----END-----